

fake_depth2 — Command Guide

English reference for every command used to build, run, and test the fake_depth2 ROS 2 project (Mini-Project A + B).

1. What This Project Is

fake_depth2 is a ROS 2 (C++) project that simulates a depth sensor for an underwater/diving vehicle. It is split into two stages that build on each other:

- **Mini-Project A** – a publisher node (depth_sensor_node) that simulates depth readings (sine wave or random) on the /depth topic, and a subscriber node (depth_monitor_node) that logs a warning the moment the reading crosses a configurable threshold.
- **Mini-Project B** – adds a **Service** (/calibrate_depth) to reset or set the sensor's zero-offset, and an **Action** (/dive_to_depth) that simulates diving to a target depth over time with live feedback and cancel support. Both nodes are then started together from one launch file.

Along the way the project also defines three **custom interfaces** (DepthReading.msg, CalibrateDepth.srv, DiveToDepth.action) in their own package, and exposes tunable **Parameters** on both nodes that can be read and changed at runtime.

2. Build the Workspace

Run these once from the workspace root (the fake_depth2 folder that contains the src/ directory), and again only after you change C++ code, interface files, or CMakeLists.txt.

rosdep install --from-paths src --ignore-src -r -y

Scans every package under src/ and installs any missing system dependencies (roscpp, roscpp_action, rosidl_generators, etc.). Run this once, the first time you set up the workspace — not before every build. Requires rosdep update to have been run at least once on the machine.

colcon build --symlink-install

Actually compiles the project. It builds fake_depth2_interfaces first (so the custom message/service/action C++ headers exist), then builds fake_depth2 against them. --symlink-install links non-compiled files (launch files, YAML params) instead of copying them, so editing those later doesn't require a full rebuild.

source install/setup.bash

Registers this workspace's packages (fake_depth2, fake_depth2_interfaces) with ROS 2 — but only in the current terminal session. Every new terminal needs this run again before ros2 run or ros2 launch will recognize the packages. (This is why Package 'fake_depth2' not found shows up if you skip it in a new tab.)

Tip: to avoid repeating this every time, append it to ~/.bashrc so every new terminal sources it automatically:

```
echo "source /opt/ros/&lt;distro&gt;/setup.bash" &gt;&gt; ~/.bashrc
echo "source ~/fake_depth2/install/setup.bash" &gt;&gt; ~/.bashrc
```

3. Run Everything With the Launch File

The normal way to run the project: one command, one terminal, both nodes started together, both configured from `config/params.yaml`.

```
ros2 launch fake_depth2 fake_depth2.launch.py
```

Override the topic name used by both nodes, without editing any file (uses the remapping wired into the launch file):

```
ros2 launch fake_depth2 fake_depth2.launch.py depth_topic:=depth_raw
```

Point the launch file at a different parameter YAML file, e.g. to test different threshold/amplitude/rate values without touching the original:

```
ros2 launch fake_depth2 fake_depth2.launch.py  
params_file:=/absolute/path/to/custom_params.yaml
```

4. Run Nodes Individually (for testing each piece)

Useful when debugging one node at a time, or watching one node's log output in isolation.

```
# Terminal 1 – sensor only  
ros2 run fake_depth2 depth_sensor_node --ros-args --params-file  
src/fake_depth2/config/params.yaml
```

Starts only the simulated depth publisher, loading the same parameter file the launch file would use.

```
# Terminal 2 – monitor only  
ros2 run fake_depth2 depth_monitor_node --ros-args --params-file  
src/fake_depth2/config/params.yaml
```

Starts only the threshold-warning subscriber, in its own terminal so its log output doesn't mix with the sensor's.

```
# Terminal 3 – watch the raw topic  
ros2 topic echo /depth
```

Prints every message published on `/depth` as it happens (`depth`, `zero_offset`, `threshold_crossed`) — the fastest way to confirm the sensor is actually publishing sane numbers before trusting the monitor's warnings.

5. Read / Change Parameters While Running

```
ros2 param get /depth_sensor_node warn_threshold
```

Reads the current value of `warn_threshold` on the sensor node *without stopping it*.

```
ros2 param set /depth_monitor_node depth_threshold 15.0
```

Changes the monitor's threshold to 15.0 *live*, with no restart needed. This works because both nodes register an `on_set_parameters` callback in code, which is what makes the new value take effect immediately.

6. Try the Service (Calibration)

The `/calibrate_depth` Service is synchronous: you send a request and get an immediate response back (success, message, applied offset).

```
ros2 run fake_depth2 calibrate_client --reset
```

Resets the sensor's zero-offset back to 0.0.

```
ros2 run fake_depth2 calibrate_client 2.5
```

Sets the zero-offset to 2.5 meters.

Both of the above use the small custom CLI client built for this project. You can call the same Service directly with the generic ROS 2 command instead:

```
ros2 service call /calibrate_depth fake_depth2_interfaces/srv/CalibrateDepth  
"{reset_to_zero: false, requested_offset: 2.5}"
```

Note: keep this on a single line. A trailing backslash (line continuation) followed by a stray second line is what caused the earlier "The passed service type is invalid" error — the shell split the command incorrectly.

7. Try the Action (Dive Simulation)

The `/dive_to_depth` Action is long-running: instead of one instant reply, you get continuous feedback while it works, and you can cancel it mid-way.

```
ros2 run fake_depth2 dive_client 25.0
```

Dives to 25 meters at the default dive rate.

```
ros2 run fake_depth2 dive_client 3.0 1.0
```

Dives to 3 meters at 1.0 m/s. Press Ctrl+C partway through to see the goal get canceled cleanly.

Equivalent generic command, which also prints feedback as it arrives:

```
ros2 action send_goal /dive_to_depth fake_depth2_interfaces/action/DiveToDepth  
"{target_depth: 20.0, dive_rate: 1.0}" --feedback
```

While a dive is in progress, `/depth` keeps publishing readings that track the simulated dive instead of the idle sine/random pattern, then resumes normal simulation once the goal finishes or is canceled.

8. Troubleshooting Notes

Symptom	Cause / Fix
Package 'fake_depth2' not found	The current terminal never ran <code>source install/setup.bash</code> . Sourcing only applies to the terminal session it was run in.
ament_cmake not found during colcon build	ROS 2 itself wasn't sourced (<code>/opt/ros/<distro>/setup.bash</code>) before building.

The passed service/action type is invalid	Usually a malformed command line (e.g. a stray line break from a trailing backslash). Re-run the full command on one line.
Sensor node not responding / not publishing on its topics	Resolved in this project's case by purging the ROS 2 installation and reinstalling it cleanly.